

★★★ 반드시 내 것으로 ★★★

#MUSTHAVE

실무 중심 이론, 실무 중심 프로젝트로 단단하게 시작하자

성낙현의 JSP 자바 웹 프로그래밍



우리는
가치가 성장하는
시간을
만듭니다.



[1단계] 빠르게 익히는 JSP 기초

00 개발 환경 구축

- 01 JSP 기본
- 02 내장 객체(Implicit Object)
- 03 내장 객체의 영역(Scope)
- 04 쿠키(Cookie)
- 05 데이터베이스
- 06 세션(Session)
- 07 액션 태그(Action Tag)
- [Project] 08 모델1 방식의 회원제 게시판 만들기
- [Project] 09 게시판에 페이징 기능 넣기

[2단계] 고급 기능으로 스킬 레벨업

- 10 표현 언어(EL : Expression Language)
- 11 JSP 표준 태그 라이브러리(JSTL)
- 12 파일 업로드 및 다운로드
- 13 서블릿(Servlet)
- [Project] 14 모델2 방식(MVC 패턴)의 자료실형 게시판 만들기

[3단계] 프로젝트로 익히는 현업 스킬

- [Project] 15 웹소켓으로 채팅 프로그램 만들기
- [Project] 16 SMTP를 활용한 이메일 전송하기
- [Project] 17 네이버 검색 API를 활용한 검색 결과 출력하기
- [Project] 18 배포하기

Chapter

02

내장 객체

(Implicit Object)

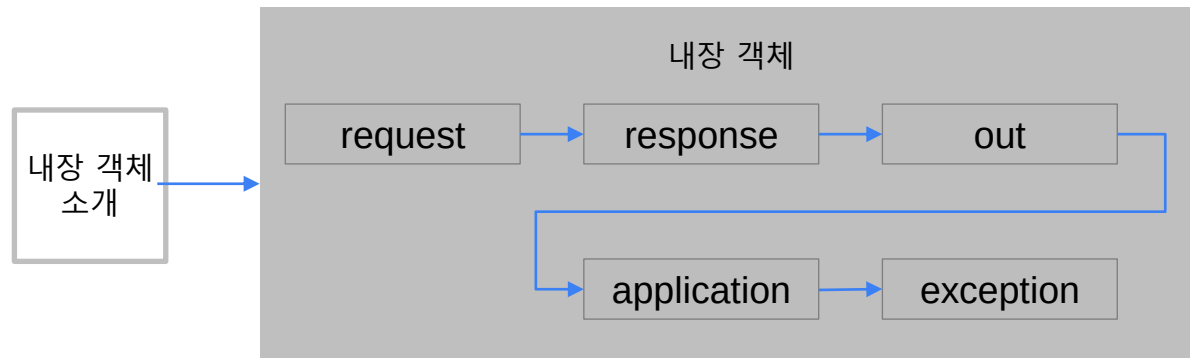


□ 학습 목표

□ 학습 순서

□ 활용 사례

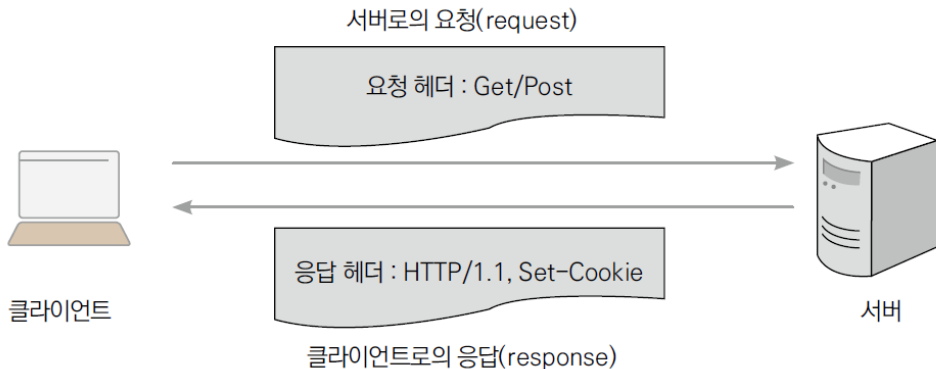
클라이언트의 요청을 받거나 응답할 때 사용되는 JSP의 기본 내장 객체들의 종류와 사용법 학습



- 웹 프로그래밍의 근간이 되는 요청과 응답부터 출력, 세션, 페이지와 애플리케이션 등 없어서는 안 될 개념들을 내장 객체로 제공하므로 수시로 광범위하게 활용

2.1 내장 객체란?(1)

- 내장 객체의 특징
 - 컨테이너가 미리 선언해놓은 참조 변수를 이용해 사용
 - 별도의 객체 생성 없이 각 내장 객체의 메서드를 사용할 수 있음
 - JSP 문서 안의 `<% 스크립틀릿 %>`과 `<%= 표현식 %>`에서만 사용할 수 있음
 - `<%! 선언부 %>`에서는 즉시 사용은 불가능하고, 매개변수로 전달받아 사용할 수는 있음

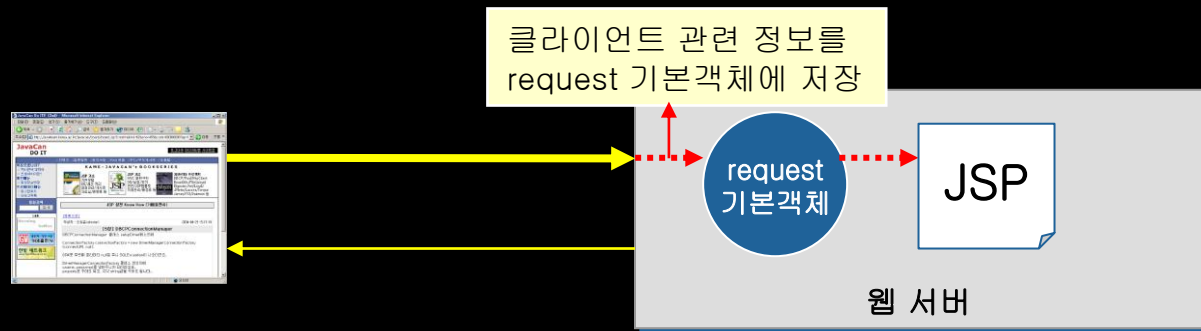


2.1 내장 객체란?(2)

- 내장 객체의 종류

내장 객체	타입	설명
request	javax.servlet.http. HttpServletRequest	클라이언트의 요청 정보를 저장
response	javax.servlet.http. HttpServletResponse	클라이언트의 요청에 대한 응답 정보를 저장
out	javax.servlet.jsp. JspWriter	JSP 페이지에 출력할 내용을 담는 출력 스트림
session	javax.servlet.http. HttpSession	웹 브라우저 정보를 유지하기 위한 세션 정보를 저장
application	javax.servlet. ServletContext	웹 애플리케이션 관련 컨텍스트 정보를 저장
pageContext	javax.servlet.jsp. PageContext	JSP 페이지에 대한 정보를 저장
page	java.lang. Object	JSP 페이지를 구현한 자바 클래스의 인스턴스
config	javax.servlet. ServletConfig	JSP 페이지에 대한 설정 정보를 저장
exception	java.lang. Throwable	예외가 발생한 경우에 사용

request 기본 객체

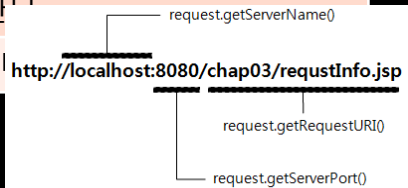


request 기본 객체가 제공하는 기능

- 클라이언트(웹 브라우저)와 관련된 정보 읽기 기능
- 서버와 관련된 정보 읽기 기능
- 클라이언트가 전송한 요청 파라미터 읽기 기능
- 클라이언트가 전송한 요청 헤더 읽기 기능
- 클라이언트가 전송한 쿠키 읽기 기능
- 속성 처리 기능

request 객체 주요메서드

메서드	리턴타입	설명
getParameter(String name)	String	이름이 name인 파라미터의 값을 구한다. 존재하지 않을 경우 null
getParameterValues(String name)	String[]	이름이 name인 모든 파라미터의 값을 배열로 구한다. 존재하지 않을 경우 null
getParameterNames()	java.util.Enumeration	웹 브라우저가 전송한 파라미터의 이름을 구한다
getParameterMap()	java.util.Map	웹 브라우저가 전송한 파라미터의 맵 <파라미터 이름, 값> 세트로 구한다
getRemoteAddr()	String	웹 서버에 연결한 클라이언트의 IP 주소를 구한다. 게시판이나 방명록 등에서 글 작성자의 IP 주소가 자동으로 입력되기도 하는데, 이때 입력되는 IP 주소가 바로 이 메서드를 사용하여 구한 것이다.
getMethod()	String	웹 브라우저가 정보를 전송할 때 사용한 방식을 구한다.
getRequestURI()	String	웹 브라우저가 요청한 URL에서 경로를 구한다.
getContextPath()	String	JSP 페이지가 속한 웹 어플리케이션의 컨텍스트 경로를 구한다.
getServerName()	String	연결할 때 사용한 서버 이름을 구한다.
getServerPort()	int	서버가 실행중인 포트 번호를 구한다.



2.2 request 객체(1)

- request 내장 객체는 JSP에서 가장 많이 사용되는 객체로, 클라이언트(주로 웹 브라우저)가 전송한 요청 정보를 담고 있는 객체
 - 클라이언트와 서버에 대한 정보 읽기
 - 클라이언트가 전송한 요청 매개변수에 대한 정보 읽기
 - 요청 헤더 및 쿠키 정보 읽기

2.2 request 객체(2)

2.2.1 클라이언트와 서버의 환경정보 읽기

- request 내장 객체를 통해 클라이언트와 서버의 환경정보 알아내기
 - [예제 2-1] 요청 페이지
 - 클라이언트의 요청에 따른 서버의 환경정보를 읽기 위해 링크를 생성
 - <a> 태그로 만든 링크이므로 GET 방식으로 전송되고, 링크 뒤에는 2개의 매개변수가 쿼리스트링으로 전달
 - <form> 태그를 사용하여 POST 방식으로 요청을 전송. 다양한 <input> 태그를 사용
 - HTTP 요청 헤더를 읽기 위한 링크를 생성

2.2 request 객체(3)

- 'GET 방식 전송' 링크나 [POST 방식 전송] 버튼을 클릭 때 나타나는 페이지 소스
 - [예제 2-2] 환경정보 읽기
 - `getMethod()` 메서드는 GET과 POST 같은 전송 방식을 반환
 - `getRequestURL()`과 `getRequestURI()` 메서드는 요청 주소를 반환
 - `getRemoteAddr()` 메서드는 클라이언트의 IP 주소를 반환
 - `getQueryString()`는 요청 주소 뒷부분의 매개변수 전달을 위한 쿼리스트링 전체를 반환
 - 쿼리스트링 중 특정 키값을 얻어오려면 `getParameter()` 메서드에 키값을 인수로 입력



- ① POST 방식이므로 주소표시줄에는 경로 외에는 아무것도 표시되지 않음
- ② 쿼리스트링은 null이 출력
- ③ 전송된 값이 한글인 경우 깨져서 출력

2.2 request 객체(4)

2.2.2 클라이언트의 요청 매개변수 읽기

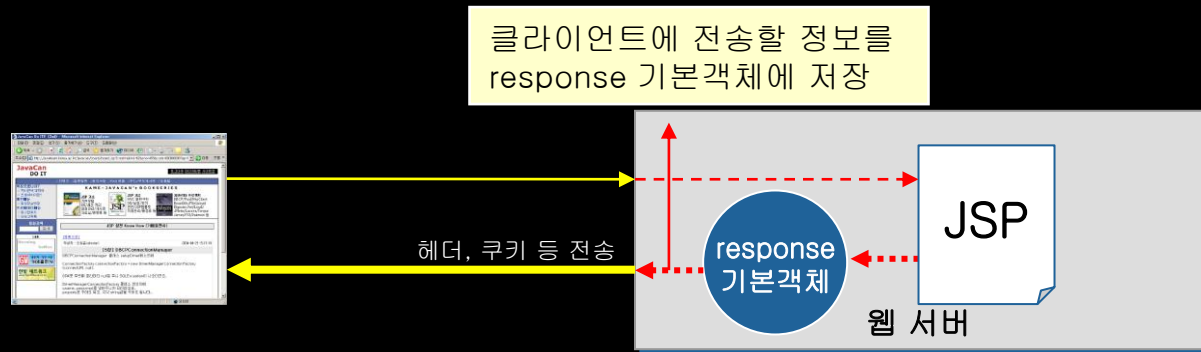
- [예제 2-3] 요청 매개변수 읽기
 - POST 방식으로 전송된 값이 한글인 경우 깨져서 출력될 때는 다국어 지원을 UTF-8로 인코딩
 - 전송되는 값이 하나라면 `getParameter( )` 메서드로 받을 수 있음. 주로 `type` 속성이 `text`, `radio`, `password`, 선택값이 하나인 `checkbox` 경우 등
 - 여러 개 선택에 사용되는 `type` 속성이 `checkbox`는 `getParameterValues( )`로 받음
값이 2개 이상이므로 `String` 배열을 반환
 - `for`문을 이용해서 `String` 배열에 담긴 값들을 하나의 문자열로 합침
 - `textarea` 태그는 텍스트를 여러 줄 입력
출력 시에는 `[enter]` 키를 `<br>` 태그로 변환해야 줄바꿈이 제대로 반영

2.2 request 객체(5)

2.2.3 HTTP 요청 헤더 정보 읽기

- [예제 2-4] 요청 헤더 읽기
 - `getHeaderNames()` 메서드는 모든 요청 헤더의 이름을 반환. 반환 타입은 Enumeration
 - while문에서 `hasMoreElements()`를 이용해 출력할 요청 헤더명이 더 있는지 확인
 - 헤더명이 더 있다면 요청 헤더의 이름을 획득
 - `getHeader()` 메서드에 헤더명을 건네 헤더값을 얻어옴
- `user-agent` : 웹 브라우저의 종류를 알 수 있음. 크롬, 파이어폭스, 익스플로러 등 여러 가지 웹 브라우저에서 테스트해보면 조금씩 다른 결과가 출력
- `referer` : 리퍼러는 웹을 서핑하면서 링크를 통해 다른 사이트로 방문 시 남는 흔적
- `cookie` : 요청 헤더를 통해 쿠키도 확인 가능 (4장 '쿠키'에서 자세히 학습)

response 기본 객체



response 기본 객체가 제공하는 기능

- 헤더 정보 전달
- 쿠키 전달
- 리다이렉트 지정

* 이외에 몇가지 기능이 더 있지만 거의 사용되지 않는다.

03. response

■ response 내장객체

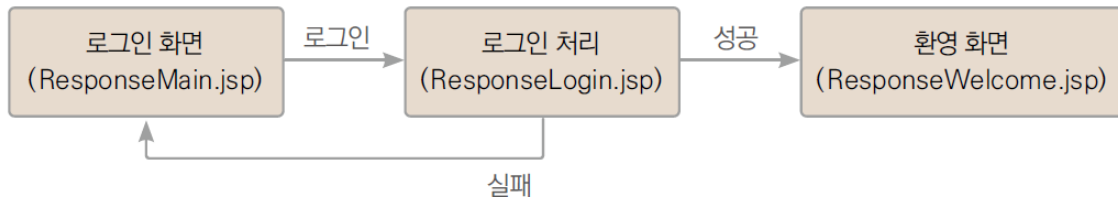
- response는 request와 반대되는 개념으로, 사용자 응답과 관련된 기능을 제공.
- 사용자 요청(request)을 처리하고 응답을 다른 페이지로 전달하는 등의 기능을 제공한다.
- javax.servlet.http.HttpServletResponse 객체에 대한 참조 변수로, request에 만큼 많이 사용되지는 않으나 setContentType, sendRedirect와 같은 메서드는 잘 알아두어야 한다.

[표 6-3] response 주요 메서드

메서드	설명
setContentType(type)	문자열 형태의 type에 지정된 MIME Type으로 contentType을 설정한다.
setHeader(name,value)	문자열 name의 이름으로 문자열 value의 값을 헤더로 세팅한다.
setDateHeader(name, date)	문자열 name의 이름으로 date에 설정된 밀리세컨드 시간 값을 헤더에 설정한다.
sendError(status,msg)	오류 코드를 세팅하고 메시지를 보낸다.
sendRedirect(url)	클라이언트 요청을 다른 페이지로 보낸다.

2.3 response 객체(1)

2.3.1 sendRedirect()로 페이지 이동하기



- [예제 2-5] 로그인 폼과 응답 헤더 설정 페이지
 - loginErr은 로그인 실패 여부를 알려주는 매개변수
이 매개변수에 값이 들어 있다면 "로그인 실패"를 출력
 - 아이디와 패스워드를 입력받는 간단한 로그인 폼을 구성
 - 응답 헤더 추가를 위한 입력 폼을 구성
 - 헤더에 추가할 데이터의 형식별로 value 속성에 미리 입력

2.3 response 객체(2)

- [예제 2-6] 로그인 처리하기
 - request 내장 객체로 전송된 매개변수를 얻어옴
 - 회원 인증을 진행
 - 인증에 성공하면 ResponseWelcome.jsp이 실행되며, sendRedirect() 메서드에 건넨 응답 페이지로 이동
 - 인증에 실패하면 request 내장 객체를 통해 로그인 페이지, 즉 ResponseMain.jsp로 포워드. 매개 변수 loginErr가 전달되고 로그인 실패 메시지를 화면에 출력
- [예제 2-7] 로그인 성공 페이지
 - 특별한 로직 없이 성공 여부만 간단히 보여줌

2.3 response 객체(3)

2.3.2 HTTP 헤더에 응답 헤더 추가하기

- add 계열은 헤더값을 새로 추가할 때 사용하고, set 계열은 기존의 헤더를 수정할 때 사용
- [예제 2-8] 응답 헤더에 값 추가하기
 - 응답 헤더에 추가할 값들을 준비하는 코드
 - 0000-00-00(년-월-일) 형식으로 전송된 add_date 매개변수의 값을 long 타입으로 변경 (타임스탬프: 1970년 1월 1일 0시 0분 0초부터 현재까지의 시간을 밀리초 단위로 환산)
 - 폼값으로 전송되는 값은 항상 String 타입이므로 add_int도 문자열로 얻어지므로 이를 정수 형태로 사용할 때는 반드시 변환
 - 응답 헤더에 값을 추가하는 코드
 - add 계열의 메서드로 헤더값을 추가
 - set 계열의 메서드를 사용하면 이전 값이 수정

forward 액션과 response.sendRedirect() 의 차이점

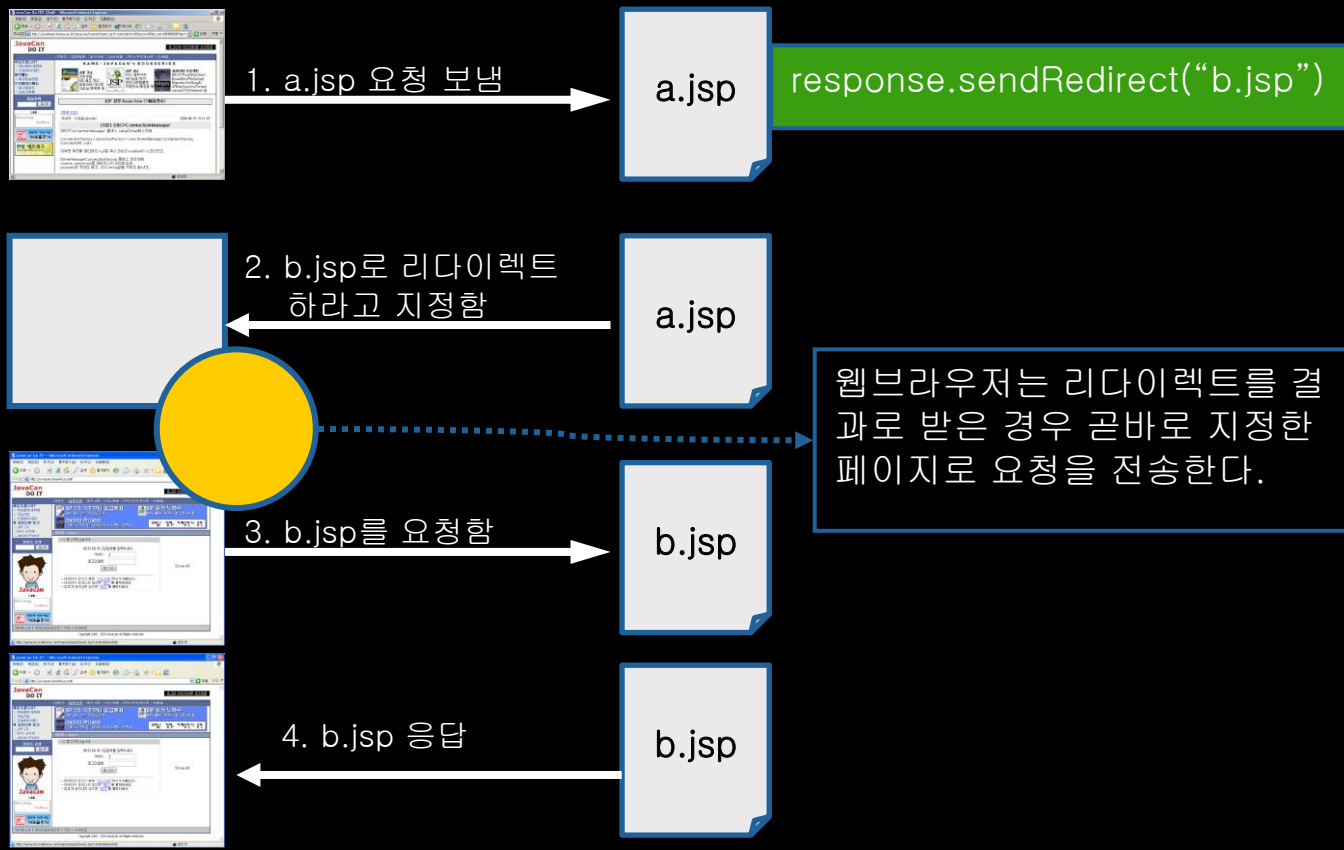
```
01 <%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
02 <% request.setCharacterEncoding("UTF-8"); %>
03     <jsp:forward page="page_control_end.jsp">
04         <jsp:param name="tel" value="000-000-0000" />
05 </jsp:forward>
```

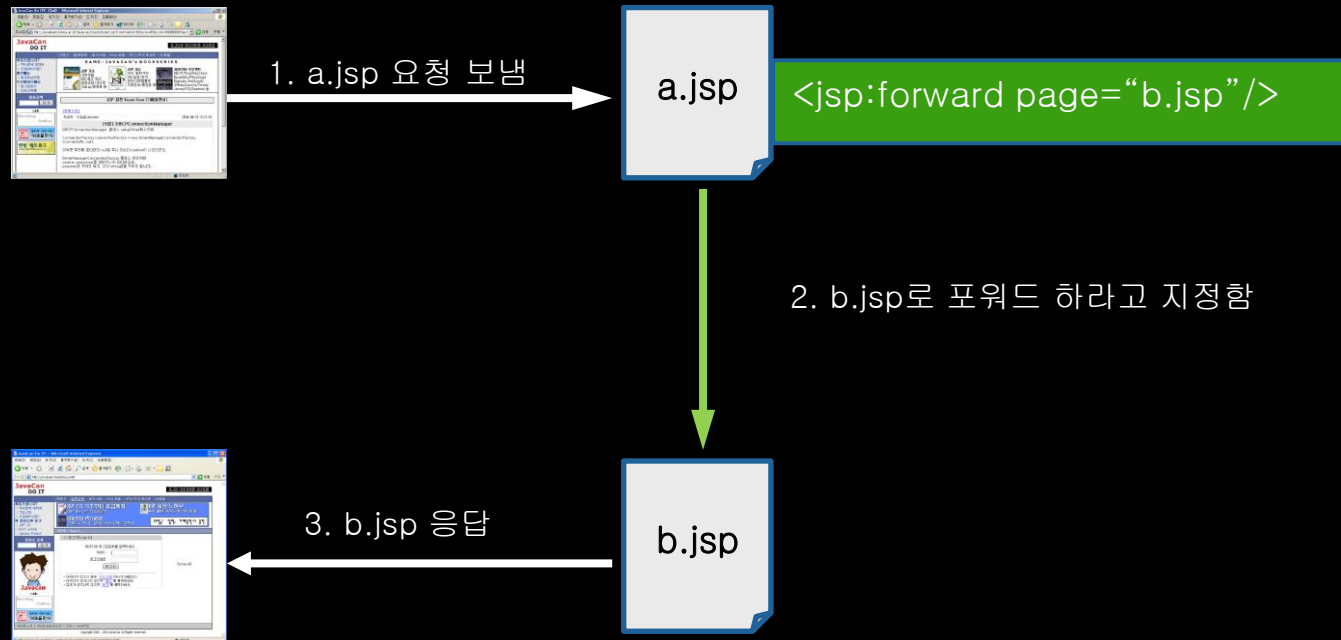
•

```
01 <% response.sendRedirect("page_control_end.jsp"); %>
```

response 기본 객체를 이용한 *redirect*

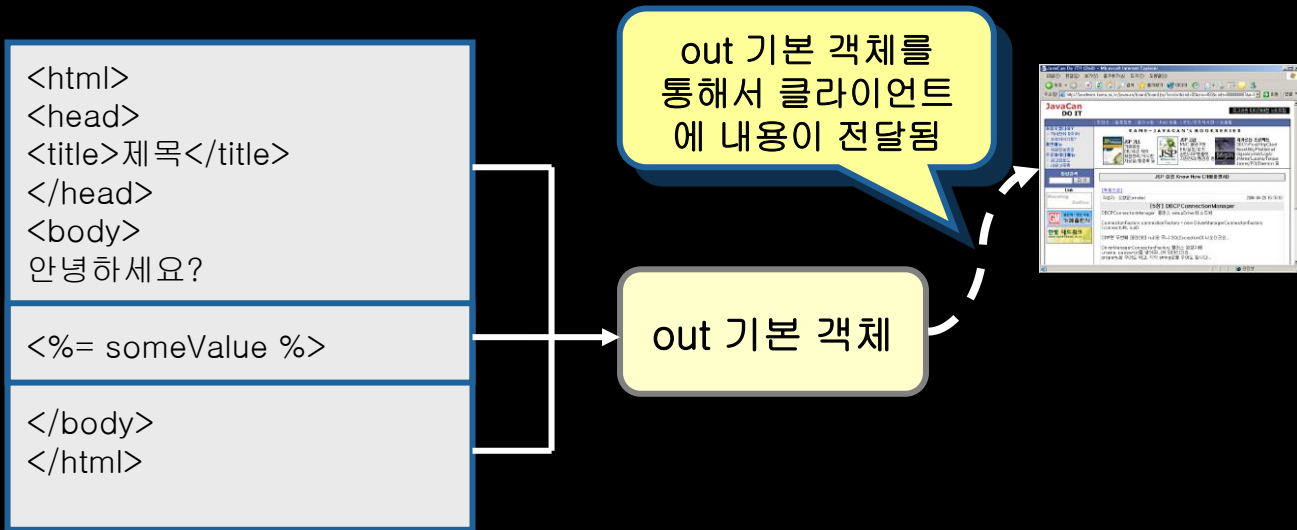
"웹 브라우저에게 다른 페이지로 이동하라고 지시하는 것을 리다이렉트(redirect)라고 한다"





out 기본 객체

"JSP 페이지가 생성하는 모든 내용은 out 기본 객체를 통해서 전송된다"



■ out 내장객체

- **out은 출력 스트림으로써, 사용자 웹 브라우저로 출력하기 위한 내장 객체임.**
- 스크립트릿에서 브라우저로 텍스트를 출력하는 데 사용.
- out은 javax.servlet.jsp.JspWriter 객체의 참조 변수로, 버퍼 관련 메서드와 출력 관련 메서드로 구성되어 out을 이용해서 출력한 내용은 서버의 콘솔이 아닌 사용자에게 전달된다.

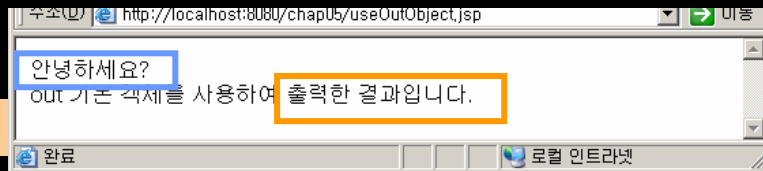
[표 6-4] out 주요 메서드

메서드	설명
getBufferSize()	output buffer의 크기를 바이트로 알려준다.
getRemaining()	남아 있는 버퍼의 크기 중 사용 가능한 비율을 알려준다.
clearBuffer()	버퍼에 있는 콘텐츠를 모두 지운다.
flush()	버퍼를 비우고 output stream도 비운다.
close()	output stream을 닫고 버퍼를 비운다.
println(content)	content의 내용을 newline과 함께 출력한다.
print(content)	content의 내용을 출력한다.

out 기본 객체의 출력 메소드

Example

```
<%@ page contentType = "text/html; charset=euc-kr" %>
<html>
<head><title>out 기본 객체 사용</title></head>
<body>
<%
    out.println("안녕하세요?");
%>
<br>
out 기본 객체를 사용하여
<%
    out.println("출력한 결과입니다.");
%>
</body></html>
```



언제 사용?

```
<% if(grade>10) { %>
<%= gradeStringA %>
<% } else if (grade>5) { %>
<%= gradeStringB %>
<% } %>
```

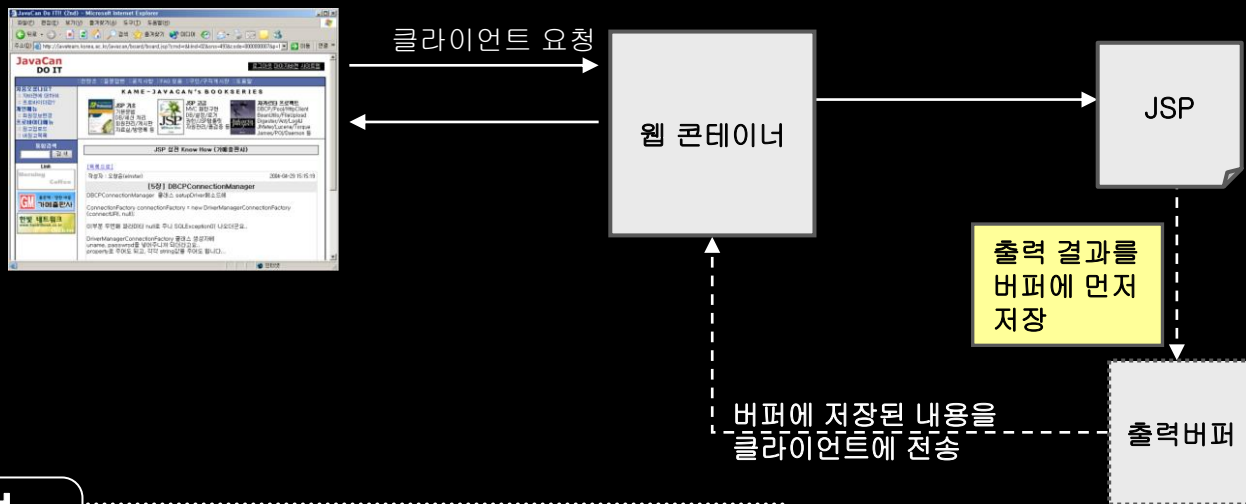


```
<%
    if(grade>10) {
        out.println(gradeStringA);
    } else if (grade>5) {
        out.println(gradeStringB);
    }
%>
```

2.4 out 객체

- out 내장 객체는 웹 브라우저에 변수 등의 값을 출력할 때 주로 사용
- out 내장 객체는 버퍼를 사용. 즉, 출력되는 모든 정보는 버퍼에 먼저 저장된 후 웹 브라우저에 출력
- [예제 2-9] out 객체로 값 출력하기
 - print() 메서드로 쓴 내용은 먼저 버퍼에 들어감
 - clearBuffer() 메서드로 버퍼 안의 내용을 삭제
 - getBufferSize()는 현재 페이지에 설정된 버퍼의 크기를 가져옴(기본값 8KB)
 - getRemaining() 메서드는 사용하고 남은 버퍼의 크기를 출력
 - flush() 메서드는 버퍼에 담긴 내용을 강제로 플러시
 - print() 메서드를 이용해 다양한 타입의 데이터를 출력

출력 버퍼



버퍼의 장점

- 출력 성능의 향상
- 중간에 버퍼를 비우고 새로운 내용을 출력할 수 있음
- 버퍼가 다 차기 전까지 헤더를 변경할 수 있다.

2.5 application 객체

- application 내장 객체는 웹 애플리케이션당 하나만 생성되며, 모든 JSP 페이지에서 접근 가능
- web.xml에 설정한 컨텍스트 초기화 매개변수를 읽어오고 폴더의 물리적 경로 찾기
- [예제 2-10] 초기화 매개변수 추가하기
 - <context-param> 태그를 만들고, 그 안에 초기화 매개변수의 이름과 값을 입력
- [예제 2-11] 다양한 서버 정보 얻어오기
 - getInitParameter() 메서드를 이용해서 web.xml에 설정한 초기화 매개변수 읽기
 - getRealPath() 메서드로 현재 예제를 작성 중인 폴더("/02ImplicitObject")의 물리적 경로를 얻어와서 출력
 - 선언부에서 application 내장 객체 사용
 - this 사용
 - 내장 객체를 인수로 전달

2.6 exception 객체(1)

- HTTP 에러 코드 설명 및 조치 방법

HTTP 에러 코드	에러 의미	조치 방법
404	Not Found : 클라이언트가 요청한 경로에서 문서를 찾을 수 없음	URL 상의 경로명이나 파일명이 제대로 입력되었는지 확인
405	Method Not Allowed : 허용되지 않는 메서드라는 뜻으로, GET 방식 혹은 POST 방식으로 요청했는데 이를 처리할 컨트롤러가 없음	주로 서블릿으로 개발할 때 발생하게 되는데 doGet(), doPost() 메서드가 적절히 오버라이딩되었는지 확인
500	Internal Server Error : 서버 내부 오류로, 코드에 오타가 있거나 로직에 문제가 있어 웹 서버가 요청 사항을 수행할 수 없음	가장 많이 발생하는 에러, 개발 중인 코드를 전반적으로 확인하여 오류를 처리

2.6 exception 객체(2)

- 에러가 발생했을 때 에러별로 출력할 페이지를 분기하는 방법
 - web.xml에 설정 추가 필요
 - [예제 2-12] web.xml에 에러별 출력 페이지 설정
 - 에러 코드를 정수로 명시
 - 해당 에러 발생 시 웹 브라우저에 출력할 페이지와 경로를 지정. <location> 엘리먼트에 경로를 명시할 때는 컨텍스트 루트를 제외한 나머지를 기술
 - 경로는 슬래시(/)로 시작
 - 편의상 Exception.jsp라는 파일 하나에서 모든 에러를 처리
 - [예제 2-13] 에러 출력 페이지
 - response 내장 객체로부터 에러 코드 확인
 - 에러 코드에 따라 적절한 메시지 출력

학습 마무리

핵심 요약

- request 객체 : 클라이언트의 요청을 받거나 웹 브라우저에 대한 정보 혹은 요청 헤더에 대한 정보를 읽을 때 사용
- response 객체 : 요청에 대한 응답을 웹 브라우저로 보낼 때 사용. 페이지 이동이나 응답 헤더를 추가할 때도 사용
- out 객체 : 변수 등의 값을 웹 브라우저에 출력할 때 주로 사용
- application 객체 : 웹 애플리케이션을 구성하는 모든 JSP에서 접근 가능한 객체로, 웹 애플리케이션에 대한 설정값을 저장할 때 주로 사용
- exception 객체 : 예외 처리를 위해 사용